

WEEK 3 · LIVE SESSION



From LLM Calls to AI Workflows

Today we stop asking AI questions and start making AI do work - step by step

LangChain Recap · Translate → Summarize · Prompt Design · LCEL · Debugging



Input



Translate



Summarize



Output

Learn with Sarvesh · Courses. Clarity. Confidence.

WHERE YOU ARE

What You Already Know

Two weeks ago these words were confusing. Now you have actually used them.

1


LLM API calls

Sending a request and getting a response back.

Like ordering on Swiggy - you send a request, you get food.

2


Prompting

How you ask determines what you get.

Asking a friend clearly vs vaguely. Same friend, different answer.

3


Temperature

The model's personality dial.

Low → serious and predictable. High → creative and playful.

4


Tokenization

How AI actually reads text.

It doesn't see whole sentences. It breaks them into pieces.

5


Embeddings

How AI understands meaning.

It knows 'cat' and 'dog' are related, without being told.



ASK THE ROOM How many of you actually ran the code from last week? Put a 1 in the chat if you did.

THIS SESSION

What We'll Do Today

This is not a heavy theory session. Today is a BUILDING session.

1



Recap LangChain

Not deep theory - just enough to get everyone on the same page.

2



Build Translate → Summarize

A real working workflow. Two steps, one automatic flow.

3



Improve prompts

Same AI, different results - depending entirely on how you ask.

4



Debug errors

Real developers don't just write code. They fix it when it breaks.

5



Q&A

Ask anything. No question is too small.



ASK THE ROOM How many of you are excited to build something today? Let's see the chat.

PART 1

LangChain - Quick Recap

You've seen this in the recorded videos - let's refresh it together



How This Section Runs

I am not going to lecture this part. You have already watched it.

I will ask questions. You answer in the chat or unmute.

If nobody answers, I'll guide you to it - but try first.

Four Questions - You Answer, Not Me



1

What is LangChain?

Is it a model? Or is it something else?

Not a model. A framework that helps us WORK with models.



2

Why do we need it?

Think about last week - API calls again and again.

Without it, code gets messy fast once you have multiple steps.



3

What is a chain?

Order food → restaurant prepares → delivery picks → you receive.

Step 1 → Step 2 → Step 3.
Input → Process → Output.



4

The main benefit?

One word each. Put them in the chat.

Reusability. Structure. Easier workflows.



ASK THE ROOM Wait for answers before revealing. Silence is fine - give it five full seconds.

LangChain In Three Ideas

Don't overthink it. Just remember these three.

1



A wrapper over LLM APIs

Last week we wrote messages, called the model, handled the response - every single time. LangChain says: let me handle that for you.

Like Swiggy. You don't call the restaurant directly. You use a platform that simplifies it.

2



It structures your prompts

Instead of plain text strings, you get reusable templates with blanks that get filled at runtime.

Like a form. Name: _____ Language: _____
That is what PromptTemplate does.

3



It builds workflows

Instead of asking one question, you connect multiple steps. One output becomes the next input.

Like a factory. Raw material → Processing → Final product.



REMEMBER THIS LangChain = Structure + Reusability + Workflows. That one line covers most of what people ask you about it.

Why LangChain? - Both Sides

Let's be fair and look at both.

Without LangChain

Repetitive code - the same messages, API call and response handling, every single time.

Prompts scattered as raw strings, hard to manage once you have ten of them.

Now imagine a bigger project: 5 steps, 10 prompts, different inputs.

Like rewriting the same paragraph by hand instead of using a template. Tiring, and error-prone.

With LangChain

Reusable components - write it once, use it again and again.

Clean structure - everything organised step by step instead of tangled.

You assemble systems instead of writing everything from scratch.

Like LEGO blocks. Each block has a purpose. You connect them to build something bigger.



REMEMBER THIS Without LangChain you code everything manually. With LangChain you build with structure.

New Updates - LangChain Is Changing Fast

Don't memorise syntax. Understand how things connect - that survives every version bump.

1



`.invoke()` is standard

One method name across everything. Before: different methods for different things.

Before → a different remote for every TV. Now → one universal remote.

2



LCEL

LangChain Expression Language. A simple way to connect steps: Step 1 | Step 2 | Step 3.

Like water flowing through pipes. You connect the pipes, not carry the water.

3



`init_chat_model`

One function instead of remembering ChatOpenAI, ChatAnthropic and the rest. Say which model, which provider.

One doorbell for the whole building instead of one per flat.

4



Less chains, more pipelines

The mindset shifted. Stop thinking in chain classes. Think in flows.

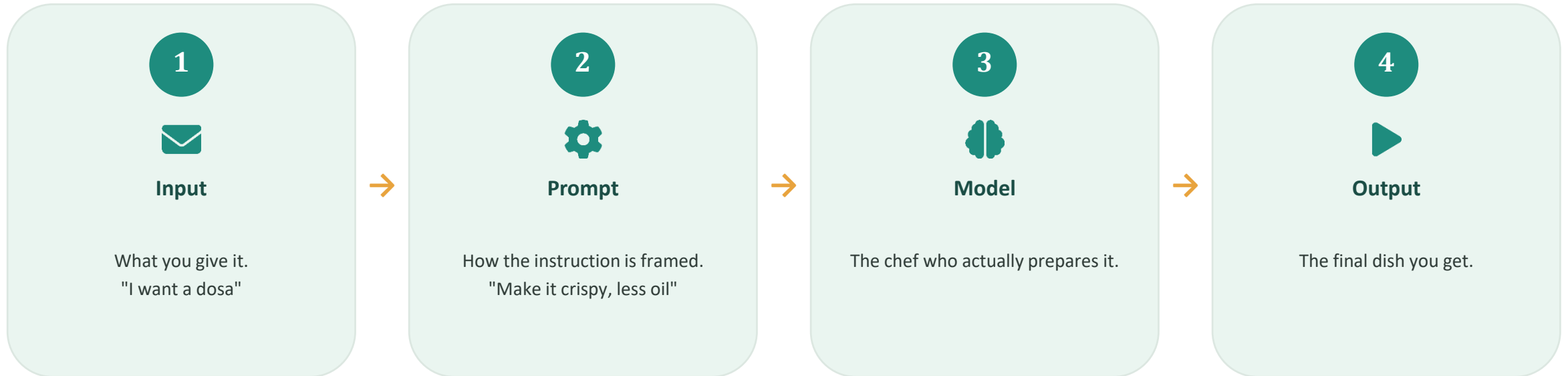
Not "I have 5 separate tasks" but "I have one flow".



REMEMBER THIS LangChain is getting simpler, more structured and more pipeline-focused - so learn the shape, not the syntax.

How LangChain Thinks

Everything in LangChain follows one flow. Learn this and the rest is detail.



Chain = connecting steps to create a flow



REMEMBER THIS Stop thinking "I will call AI once." Start thinking "how can I build a sequence of steps?"

PART 2

Today's Project

Translate → Summarize



Real-World Example: The Long Article

You get a long article in English. You'd rather read it in French or Spanish.

So you translate it. Then you don't want to read the whole thing again - you want a short version.

So you summarise it. Two steps, done by hand.

Today we make that one automatic flow.

Breakdown - Three Small Steps

Don't think of this as one big problem. Think of it as three small ones.



One input → passed through multiple steps → one final result. That is how real AI systems are built.



ASK THE ROOM If one step fails, what happens to the whole flow? Think about it before I answer.

Prompt Design - Three Things That Matter

Even with LangChain, output quality still depends entirely on how you write the prompt.

1



Clear instructions

Don't just say "translate this". Be specific about what you want.

"Translate the following text into French in a simple, natural way."

2



Role definition

Tell the model who it should behave like. It changes the whole response.

"You are a professional translator." "You are an expert summarizer."

3



Output format

Tell it how the answer should look. This is what kills inconsistency.

"Give the answer in 3 bullet points." "Give a short paragraph."



REMEMBER THIS Before writing any chain, ask three things: what do I want, how should it behave, how should the output look.

Common Mistakes

Most people don't get bad results because the model is weak. They get bad results because the prompt is weak.

1



Vague prompts

"Explain this" - the model doesn't know how detailed, for whom, or in what format. It answers anyway, just not the way you wanted.

BETTER

"Explain this in simple terms for a beginner, in 3 points."

2



No structure

Without guidance the model picks its own format. Sometimes paragraphs, sometimes lists. Never consistent.

BETTER

"Give the answer in bullet points."

3



Too long prompts

More text does not mean a better result. Long prompts confuse, cost more, and often reduce clarity.

BETTER

Clear, structured and concise beats long every time.



ASK THE ROOM Which is better - a long messy prompt, or a short clear one? Answer before I do.

PART 3

Let's Build It

Editors open - we are writing code from here



How This Part Runs

I build one step. You build the same step. Then we run it before moving on.

Do not race ahead. If something errors, say so in the chat - we fix it together.

By the end of this part, you will have a working two-stage pipeline on your own machine.

Step 0 - Setup With `init_chat_model`

setup.py

```
pip install langchain langchain-openai python-dotenv

from dotenv import load_dotenv
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

llm = init_chat_model("gpt-4o-mini",
                     model_provider="openai",
                     temperature=0)

parser = StrOutputParser()
```

-  **init_chat_model** Name the model and the provider. That's it.
-  **Swap providers** "openai" → "anthropic" and nothing else changes.
-  **temperature=0** Translation is factual. No creativity wanted.
-  **StrOutputParser** Pulls plain text out of the response object.



REMEMBER THIS `init_chat_model` is the new standard - one function instead of remembering a class name per provider.

Step 1 - The Translate Chain

step1_translate.py

```
translate_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a professional translator. "
     "Translate naturally, not word for word."),
    ("human",
     "Translate this into {language}:\n\n{text}")
])

translate = translate_prompt | llm | parser

print(translate.invoke({
    "language": "French",
    "text": "AI is changing how we work."}))
```

1



**Role in system
msg**

"Professional translator" -
role definition, in action.

2



Two variables

{language} and {text} - so
one chain handles every
language.

3



The pipe

Read the | as the word
"then".

4



Test it alone

Run this before writing
step 2. Always.



REMEMBER THIS Making {language} a variable instead of hardcoding "French" is what makes the chain reusable.

Step 2 - The Summarize Chain

step2_summarize.py

```
summarize_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are an expert summarizer. "
     "Be concise. Never add information."),
    ("human",
     "Summarize this in 3 bullet points:\n\n{content}")
])

summarize = summarize_prompt | llm | parser

print(summarize.invoke({
    "content": "<paste some text here>"
}))
```

-  **Its own persona** "Expert summarizer", not "translator". Different job.
-  **"Never add information"** A guardrail against invented context.
-  **Explicit format** "3 bullet points", not "summarize it".
-  **Test it alone too** Confirm it works before we join anything.



REMEMBER THIS Every stage gets its own system message - the persona should match the job that stage is doing.

LCEL - The Thing That Joins Them



Real-World Example: Connecting Pipes

Water flows from one pipe into the next.

You don't carry a bucket from pipe one to pipe two by hand.

You connect the pipes once, and the water finds its own way through.

LCEL is that connection. You define the flow, not the plumbing between each step.

Is LCEL doing something new? No. It is making the same logic simpler and cleaner.



Less code

No manually passing outputs into the next function call.



Cleaner structure

The code reads like the diagram: input, translate, summarize, output.



Free superpowers

Every LCEL chain gets `.invoke`, `.batch` and `.stream` automatically.



REMEMBER THIS LCEL = an easy way to connect steps. Same logic you'd write by hand, far less code to write it.

Step 3 - Combine Into One Pipeline

OPTION A · summary only

```
pipeline = (
    translate
    | (lambda text: {"content": text})
    | summarize
)

print(pipeline.invoke({
    "language": "French",
    "text": article}))
```

OPTION B · keep both

```
both = translate | (lambda t: {
    "translation": t,
    "summary": summarize.invoke({"content": t})})
```



The lambda adapts

translate gives a string; summarize wants {content}.



The classic bug

Forget it and you get a type error. Everyone hits it.



Option A or B

Summary only, or keep the translation too. That's Step 3.



Read it top down

The code now matches the diagram we drew.



REMEMBER THIS The lambda between two chains is doing real work - it reshapes one step's output into the next step's input.

Run It - Then Change One Thing At A Time

The build is done. Now the learning starts.
RUN

```
article = ""<paste a paragraph>""

print(pipeline.invoke({"language": "Spanish",
                      "text": article}))
```

THE RULE FOR EXPERIMENTS

Change **ONE** thing. Run it. Look at the difference.

Then change it back before you change the next thing.

Change three things at once and you learn nothing - you won't know which one did it.

This is how you build prompt intuition.

- 1**  **Change the language** Spanish, Tamil, German. One word changed, whole chain still works.
- 2**  **Change the format** "3 bullet points" → "one sentence" → "a tweet". Watch it obey.
- 3**  **Reverse the order** Summarize first, then translate. Cheaper. Slightly less accurate. Try both.
- 4**  **Stream it** Swap .invoke for .stream and watch the bullets appear live.
- 5**  **Remove the role** Delete "professional translator" and compare. This one surprises people.



REMEMBER THIS Change one thing, run, compare, revert. That loop is how prompt intuition actually gets built.

Debugging - The Four You'll Actually Hit

Your code will break. That is completely normal. The skill is identifying the problem quickly.

1



API key issues

Key missing, expired or wrong - the model simply won't respond.

Is it loaded? Is it correct? Is it in the .env file?

2



Wrong model name

A typo, or a model your account can't access.

Double-check the exact model string against the provider's docs.

3



Token limits

Input too long, or you asked for more output than the model allows.

Manage input size. Chunk long documents before sending them.

4



Parsing errors

You expected JSON. The model gave you a friendly paragraph.

Always validate output. Never assume the shape you asked for.



ASK THE ROOM When you get an error, what should you do FIRST? Answer before I tell you.

What You Built Today

We moved fast. Let's stop and notice what actually happened.

1



A multi-step AI workflow

Not one prompt, not one response. You connected Translate → Summarize. This is how real AI systems are built - a sequence of tasks, not one big call.

2



Prompt engineering

You saw clear instructions, role definition and output format change the result directly. Even the best model gives poor output if the prompt is unclear.

3



Debugging skills

API issues, wrong model names, unstructured outputs. Instead of getting stuck, you learned how to find and fix them. That is what makes you a developer.



REMEMBER THIS Two weeks ago you were learning what AI is. Today you built a working AI pipeline. That is a big shift.

Where We Go From Here

Today you built a fixed flow. Next we make it smarter, then we make it decide for itself.

1

RAG Systems

Instead of the model answering from memory, it uses **YOUR** data.

Ask questions on your company documents, your PDFs, your database.

2

Agents

Instead of following fixed steps, the AI decides what to do next.

"Book me a trip" → it searches flights, compares prices, suggests options. On its own.

3

Real applications

Everything you're learning is not theory. It ships.

Chatbots, document assistants, automation tools, AI-based products.



ASK THE ROOM Which are you most excited about - RAG, Agents, or building real apps? Chat it.



Thank You!

You stopped calling AI and started making AI do work. That's the shift.

Next up: RAG - making AI answer from YOUR data

Learn with Sarvesh · Courses. Clarity. Confidence.